

PYTHON

PARA ANÁLISE DE DADOS — NOTAS COMPLETAS

DO BÁSICO AO AVANÇADO 🚀



Fundamentos
Sintaxe · Tipos



Estruturas
Dados · Funções



NumPy
Arrays



Pandas
DataFrames



Visualização
Matplotlib ·
Seaborn



Estatística
& Cheat Sheet

FUNDAMENTOS DO PYTHON

Linguagem simples e legível — hoje é o padrão de mercado para análise de dados.

1 POR QUE PYTHON PARA DADOS?

- Sintaxe simples e legível, curva de aprendizado suave
- Ecossistema gigante: Pandas, NumPy, Matplotlib, Scikit-learn
- Integra com SQL, Excel, APIs, Machine Learning
- Comunidade enorme — fácil achar solução para tudo

2 VARIÁVEIS

```
nome = "Ana" # str idade = 28 # int altura = 1.68 # float ativo = True # bool
```

🔔 LEMBRE-SE

Python é **dinamicamente tipado** — você não declara o tipo, ele é inferido no momento da atribuição.

3 TIPOS DE DADOS BÁSICOS

TIPO	EXEMPLO
int	10, -5, 2024
float	3.14, -0.5
str	"dados", 'python'
bool	True, False
NoneType	None

4 OPERADORES

- Aritméticos — + - * / // % **
- Comparação — == != > < >= <=
- Lógicos — and, or, not

★ DICA

Use `type(variavel)` sempre que tiver dúvida sobre o tipo de um dado durante a análise.

ESTRUTURAS DE CONTROLE

Como o Python decide o que executar e quantas vezes repetir uma ação.

① CONDICIONAIS

```
if idade >= 18: print("maior de idade") elif  
idade >= 12: print("adolescente") else:  
print("criança")
```

② LAÇOS DE REPETIÇÃO

```
for item in [1, 2, 3]: print(item) i = 0  
while i < 3: print(i) i += 1
```

③ CONTROLE DENTRO DE LOOPS

- **break** — interrompe o loop imediatamente
- **continue** — pula para a próxima iteração
- **pass** — não faz nada (placeholder)

④ LIST COMPREHENSION

```
quadrados = [x**2 for x in range(5)] # [0,  
1, 4, 9, 16] pares = [x for x in range(10)  
if x % 2 == 0]
```

DICA

List comprehensions são mais **rápidas e pythônicas** que loops tradicionais para criar listas — muito usadas em preparação de dados.

ESTRUTURAS DE DADOS

As "caixas" onde o Python guarda coleções de dados — cada uma com um propósito diferente.

ESTRUTURA	SINTAXE	MUTÁVEL?	ORDENADA?	USO COMUM
Lista	[1, 2, 3]	Sim	Sim	Coleções que mudam
Tupla	(1, 2, 3)	Não	Sim	Dados fixos, retorno de funções
Dicionário	{"a": 1}	Sim	Sim (3.7+)	Pares chave-valor
Set	{1, 2, 3}	Sim	Não	Valores únicos, operações de conjunto

① LISTAS — MÉTODOS COMUNS

```
dados = [10, 20, 30] dados.append(40)
dados.remove(10) dados.sort() len(dados) # 3
dados[0] # indexação dados[1:3] # slicing
```

② DICIONÁRIOS — NA PRÁTICA

```
pessoa = { "nome": "Ana", "idade": 28 }
pessoa["idade"] # 28 pessoa.keys()
pessoa.values() pessoa.items()
```

💡 LEMBRE-SE

Dicionários são a base de dados semiestruturados em Python — JSON vira dicionário automaticamente ao ser lido.

FUNÇÕES & MÓDULOS

Organize e reutilize código — a base de qualquer script de análise bem estruturado.

① DEFININDO FUNÇÕES

```
def media(numeros): return sum(numeros) /  
len(numeros) media([10, 20, 30]) # 20.0
```

② ARGUMENTOS FLEXÍVEIS

```
def resumo(*args, **kwargs): print(args) #  
tupla print(kwargs) # dicionário resumo(1,  
2, fonte="csv")
```

③ FUNÇÕES LAMBDA

```
dobro = lambda x: x * 2 dobro(5) # 10 #  
muito usada dentro do pandas: #  
df["col"].apply(lambda x: x * 2)
```

④ IMPORTANDO MÓDULOS

```
import pandas as pd import numpy as np from  
datetime import datetime # instalar pacote:  
pip install pandas
```

★ DICA

Funções pequenas e com um único propósito facilitam **testar e reaproveitar** partes do seu pipeline de análise.

STRINGS & ARQUIVOS

Texto e arquivos são a porta de entrada da maioria dos dados do mundo real.

① MÉTODOS DE STRING ÚTEIS

```
s = " Dados,Sujos " s.strip() # remove  
espaços s.lower() # minúsculas  
s.replace(",",";") # substitui s.split(",")  
# divide em lista nome = "Ana" f"Olá,  
{nome}!" # f-string
```

② TRATAMENTO DE ERROS

```
try: valor = 10 / 0 except  
ZeroDivisionError: print("não dá pra dividir  
por zero") finally: print("fim da  
tentativa")
```

③ LENDO E ESCRIVENDO ARQUIVOS

```
with open("dados.txt") as f: conteudo =  
f.read() with open("saida.txt", "w") as f:  
f.write("resultado da análise")
```

🔗 DICA

O bloco `with` fecha o arquivo automaticamente
— evita vazamento de recursos, mesmo se der
erro no meio do caminho.

NUMPY — ARRAYS

A base numérica do ecossistema de dados em Python — arrays rápidos e vetorizados.

1 CRIANDO ARRAYS

```
import numpy as np
a = np.array([1, 2, 3, 4])
zeros = np.zeros(5)
uns = np.ones((2,3))
seq = np.arange(0, 10, 2)
lin = np.linspace(0, 1, 5)
```

3 OPERAÇÕES VETORIZADAS

```
a = np.array([1, 2, 3])
a * 2 # [2, 4, 6]
a + a # [2, 4, 6]
a.mean() # 2.0
a.sum() # 6
a.max(), a.min()
```

2 POR QUE NÃO USAR LISTAS?

- Arrays NumPy são **vetorizados** — operações aplicam a todos os elementos de uma vez
- Muito mais rápido que loops em Python puro
- Usa menos memória que listas

4 INDEXAÇÃO & SHAPE

```
m = np.array([[1,2],[3,4]])
m.shape # (2, 2)
m[0, 1] # 2
m.reshape(4,1)
```

💡 LEMBRE-SE

Pandas é construído **em cima** do NumPy — entender arrays ajuda a entender por que Series e DataFrames são tão rápidos.

PANDAS — SERIES & DATAFRAME

A ferramenta mais usada para análise de dados tabulares em Python.

1 SERIES (1 DIMENSÃO)

```
import pandas as pd  
s = pd.Series([10, 20, 30])  
s = pd.Series([10, 20], index=["a", "b"])
```

2 DATAFRAME (2 DIMENSÕES)

```
df = pd.DataFrame({ "nome": ["Ana", "Léo"],  
                    "idade": [28, 34] })
```

3 EXPLORANDO UM DATAFRAME

COMANDO	O QUE FAZ
df.head()	Primeiras 5 linhas
df.tail()	Últimas 5 linhas
df.shape	(linhas, colunas)
df.columns	Nomes das colunas
df.dtypes	Tipo de cada coluna
df.info()	Resumo geral
df.describe()	Estatísticas básicas

★ DICA

`df.head()` e `df.info()` devem ser os dois primeiros comandos que você roda ao abrir qualquer novo dataset.

IMPORTANDO & EXPORTANDO DADOS

Quase toda análise começa lendo um arquivo e termina salvando um resultado.

① LENDO DADOS

```
pd.read_csv("vendas.csv")
pd.read_excel("dados.xlsx")
pd.read_json("dados.json")
pd.read_sql(query, conexao)
```

③ SALVANDO DADOS

```
df.to_csv("saida.csv", index=False)
df.to_excel("saida.xlsx")
df.to_json("saida.json")
```

② PARÂMETROS ÚTEIS DO READ-CSV

PARÂMETRO	FUNÇÃO
sep=";"	Define o separador
encoding="utf-8"	Codificação do arquivo
index_col=0	Usa 1ª coluna como índice
usecols=[...]	Lê só colunas específicas
na_values=["-"]	Define o que é valor nulo

🔗 DICA

Use `index=False` ao salvar CSV para não criar uma coluna extra com o índice numérico do pandas.

LIMPEZA DE DADOS

Dados reais são bagunçados — limpar é a etapa que mais consome tempo em qualquer análise.

1 VALORES AUSENTES

```
df.isna().sum() # conta nulos/coluna
df.dropna() # remove linhas com nulo
df.fillna(0) # preenche com valor
df["col"].fillna(df["col"].mean())
```

2 DUPLICADOS

```
df.duplicated().sum() df.drop_duplicates()
```

3 TIPOS & RENOMEAÇÃO

```
df["idade"] = df["idade"].astype(int)
df["data"] = pd.to_datetime(df["data"])
df.rename(columns={"nm": "nome" })
```

4 CHECKLIST DE LIMPEZA

- Colunas com nomes consistentes (minúsculo, sem espaço)
- Tipos de dados corretos em cada coluna
- Nulos tratados de forma consciente
- Duplicados removidos ou justificados

💡 LEMBRE-SE

"Garbage in, garbage out" — nenhuma análise ou gráfico corrige dados mal limpos na origem.

SELEÇÃO, FILTROS & INDEXAÇÃO

Como pegar exatamente as linhas e colunas que você precisa.

① LOC VS ILOC

	BASEADO EM	EXEMPLO
loc	Rótulo/nome	df.loc[0, "nome"]
iloc	Posição numérica	df.iloc[0, 1]

③ FILTROS BOOLEANOS

```
df[df["idade"] > 30] df[(df["idade"] > 30) & (df["ativo"])] df.query("idade > 30")
```

② SELECIONANDO COLUNAS

```
df["idade"] # uma coluna (Series)  
df[["nome", "idade"]] # várias colunas (DF)
```

④ ORDENAÇÃO

```
df.sort_values("idade", ascending=False)
```

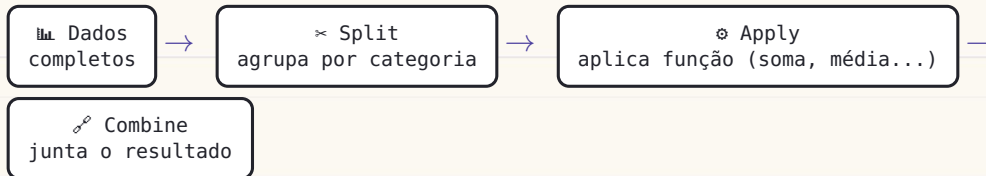
★ DICA

Use & e | (não and/or) para combinar condições em filtros booleanos do pandas — e sempre entre parênteses.

AGRUPAMENTO & AGREGAÇÃO

A ferramenta mais poderosa do pandas para resumir dados por categoria.

① SPLIT → APPLY → COMBINE



② GROUPBY NA PRÁTICA

```
df.groupby("categoria")["venda"].sum()
df.groupby("categoria").mean()
df.groupby(["cat", "regiao"]).size()
```

④ TABELAS DINÂMICAS

```
df.pivot_table( values="venda",
index="regiao", columns="mes", aggfunc="sum"
)
```

③ AGG — MÚLTIPLAS FUNÇÕES

```
df.groupby("categoria").agg({ "venda":
["sum", "mean"], "qtd": "count" })
```

DICA

`pivot_table` é o equivalente pandas da tabela dinâmica do Excel — ótimo para relatórios resumidos.

MERGE, JOIN & CONCATENAÇÃO

Combinar informações vindas de tabelas diferentes — como um JOIN de SQL.

TIPO	RESULTADO
inner	Só as linhas que existem nas duas tabelas
left	Todas da esquerda + correspondências da direita
right	Todas da direita + correspondências da esquerda
outer	Todas as linhas de ambas, com NaN onde não houver par

1 MERGE()

```
pd.merge( clientes, pedidos,  
on="cliente_id", how="left" )
```

2 CONCATO

```
# empilhar DataFrames iguais  
pd.concat([df_jan, df_fev]) # lado a lado  
pd.concat([df1, df2], axis=1)
```

💡 LEMBRE-SE

Use `merge` para **combinar por chave** (como um JOIN) e `concat` para **empilhar** tabelas com a mesma estrutura.

DATAS & SÉRIES TEMPORAIS

Praticamente todo dataset de negócio tem uma dimensão de tempo.

① CONVERTENDO PARA DATA

```
df["data"] = pd.to_datetime(df["data"])
df["data"].dt.year df["data"].dt.month
df["data"].dt.day_name()
```

③ REAMOSTRAGEM (RESAMPLE)

```
# total de vendas por mês
df.set_index("data")["venda"] \
.resample("M").sum()
```

② FILTRANDO POR PERÍODO

```
df[df["data"] >= "2026-01-01"]
df.set_index("data").loc["2026-01"]
```

④ CÓDIGOS DE FREQUÊNCIA

CÓDIGO	PERÍODO
D	Dia
W	Semana
M	Mês
Y	Ano

VISUALIZAÇÃO COM MATPLOTLIB

A biblioteca base de gráficos em Python — flexível e totalmente customizável.

① GRÁFICOS BÁSICOS

```
import matplotlib.pyplot as plt  
plt.plot(x, y) # linha  
plt.scatter(x, y) # dispersão  
plt.bar(cats, vals) # barras  
plt.hist(dados) # histograma  
plt.show()
```

② CUSTOMIZAÇÃO

```
plt.title("Vendas por Mês")  
plt.xlabel("Mês") plt.ylabel("R$")  
plt.legend(["2025", "2026"])
```

③ MÚLTIPLOS GRÁFICOS

```
fig, ax = plt.subplots(1, 2)  
ax[0].plot(x, y)  
ax[1].bar(cats, vals)  
plt.tight_layout()
```

④ QUANDO USAR CADA GRÁFICO

- Linha — evolução ao longo do tempo
- Barra — comparar categorias
- Dispersão — relação entre 2 variáveis
- Histograma — distribuição de uma variável

VISUALIZAÇÃO COM SEABORN

Construído sobre o Matplotlib — gráficos estatísticos bonitos com muito menos código.

① GRÁFICOS MAIS USADOS

```
import seaborn as sns
sns.histplot(df["idade"])
sns.boxplot(x="cat", y="venda", data=df)
sns.scatterplot(x="a", y="b", data=df)
sns.heatmap(df.corr(), annot=True)
sns.pairplot(df)
```

② MATPLOTLIB VS SEABORN

MATPLOTLIB	SEABORN
Controle total, mais código	Padrões bonitos, menos código
Gráficos genéricos	Gráficos estatísticos prontos
Base de tudo	Usa o Matplotlib por baixo

★ DICA

`sns.heatmap(df.corr(), annot=True)` é um dos gráficos mais úteis para uma primeira exploração de correlações entre variáveis.

ESTATÍSTICA & CORRELAÇÃO

Resumir e entender os dados numericamente antes (e depois) de visualizar.

① MEDIDAS CENTRAIS

```
df["venda"].mean() # média
df["venda"].median() # mediana
df["venda"].mode() # moda df["venda"].std()
# desvio padrão df["venda"].var() #
variância
```

② RESUMO COMPLETO

```
df.describe() # count, mean, std, min, #
25%, 50%, 75%, max
```

③ CORRELAÇÃO

```
df.corr() # -1 a 1: força e direção # da
relação entre variáveis
```

④ DETECTANDO OUTLIERS (IQR)

```
Q1 = df["venda"].quantile(0.25) Q3 =
df["venda"].quantile(0.75) IQR = Q3 - Q1
limite = Q3 + 1.5 * IQR
```

💡 LEMBRE-SE

Correlação **não** implica causalidade — duas variáveis correlacionadas podem apenas compartilhar uma causa em comum.

JUPYTER, SQL & AMBIENTE

As ferramentas que cercam o dia a dia de quem analisa dados em Python.

① JUPYTER NOTEBOOK

- Código executado em **células**, com saída logo abaixo
- Ótimo para explorar dados e ver gráficos junto do código
- Comandos mágicos: %time, %matplotlib inline

② AMBIENTES VIRTUAIS

```
python -m venv venv source venv/bin/activate
# Linux/Mac venv\Scripts\activate # Windows
pip install -r requirements.txt
```

③ CONECTANDO A SQL

```
import sqlite3 conexao =
sqlite3.connect("banco.db") df =
pd.read_sql( "SELECT * FROM vendas", conexao
)
```

④ BIBLIOTECAS ESSENCIAIS

BIBLIOTECA	PARA QUÊ
pandas	Manipulação de dados
numpy	Computação numérica
matplotlib/seaborn	Visualização
scikit-learn	Machine Learning

CHEAT SHEET DE ANÁLISE DE DADOS

Revisão rápida de tudo o que você precisa — de um vistão.

① PIPELINE TÍPICO DE ANÁLISE



② COMANDOS PANDAS MAIS USADOS

COMANDO	FUNÇÃO
read_csv()	Ler arquivo CSV
head() / info()	Primeira exploração
isna() / fillna()	Tratar nulos
loc[] / iloc[]	Selecionar dados
groupby()	Agrupar & agregar
merge()	Combinar tabelas
describe()	Estatísticas gerais

🔔 LEMBRE-SE

A maior parte do tempo de qualquer análise vai para **limpeza de dados**, não para os gráficos — invista tempo nessa etapa.

③ ROTEIRO DE ESTUDO SUGERIDO

1. Python básico (sintaxe, estruturas, funções)
2. NumPy — arrays e vetorização
3. Pandas — Series, DataFrame, limpeza
4. GroupBy, merge & séries temporais
5. Visualização — Matplotlib & Seaborn
6. Estatística aplicada + projetos reais

★ PRÓXIMO PASSO

Depois de dominar isso, avance para Scikit-learn (Machine Learning) e SQL avançado para consultas mais robustas.